

# Documentazione: generatore di dati sintetici di navigazione web

File di riferimento: `sintetic_data_doc`

Questo documento descrive in modo dettagliato il codice PySpark fornito. Il codice genera dati sintetici che simulano la navigazione di utenti su un sito e-commerce e salva il risultato in una tabella Delta partizionata per data. Ogni sezione descrittiva è seguita da un'appendice tecnica con il relativo estratto di codice.

## 1. Configurazione iniziale

Vengono impostati i parametri principali e la lista degli oggetti consultabili. Questa parte definisce il contesto globale della generazione.

Appendice tecnica – codice:

```
from pyspark.sql import functions as f
from pyspark.sql import Window

# =====
# Config
# =====
N_USERS = 20000
N_SESSIONS_TARGET = 100000
P_ANON = 0.25
DAYS_BACK = 365

objects = ["Zaino", "Valigia", "Portafoglio", "Cintura", "Borsone", "Portadocumenti", "Scarpe", "Portachiavi"]
objects_arr = f.array(*[f.lit(o) for o in objects])

JOURNEY_PATTERNS = [
("full_purchase", ["Home", "Search", "Category", "Product", "Cart", "Checkout"], 0.12, 5, 6, "any", "Acquisto completo con ricerca"),
("direct_purchase", ["Home", "Product", "Cart", "Checkout"], 0.06, 4, 4, "returning", "Acquisto diretto"),
...
]
```

## 2. STEP 1 – Utenti, IP e numero di visite

Si generano 20.000 utenti, ognuno con un account, un IP deterministico e un numero di visite assegnato in modo stocastico per raggiungere ~100.000 sessioni totali. Viene inoltre assegnato un tipo utente (new, returning, power).

Appendice tecnica – codice:

```
users = (
spark.range(N_USERS)
.withColumnRenamed("id", "user_idx")
.withColumn("Account", f.concat(f.lit("user"), f.lpad(f.col("user_idx").cast("string"), 6, "0")))
.withColumn("h", f.crc32(f.col("Account")).cast("bigint"))
.select(
"user_idx", "Account",
f.concat_ws(".",
f.lit(10),
((f.col("h") % 254) + 1).cast("int"),
(f.floor(f.col("h") / 254) % 254 + 1).cast("int"),
(f.floor(f.col("h") / 64516) % 254 + 1).cast("int"))
).alias("Indirizzo_IP"),
f.exp(f.randn(123)).alias("w")
)
)

sum_w = users.agg(f.sum("w").alias("S")).first()["S"]
users = (users
.withColumn("q_i", f.col("w")/f.lit(sum_w) * f.lit(N_SESSIONS_TARGET))
.select(
"user_idx", "Account", "Indirizzo_IP",
(f.floor("q_i") + f.when(f.rand(777) < (f.col("q_i") - f.floor("q_i")),
1).otherwise(0)).cast("long").alias("visite_int")
)
)

tot = users.agg(f.sum("visite_int").alias("tot")).first()["tot"]
delta = int(tot - N_SESSIONS_TARGET)
if delta != 0:
k = abs(delta)
adj = (users.select("user_idx").orderBy(f.rand(999)).limit(k).withColumn("bump", f.lit(-1 if delta > 0 else 1)))
users = (users.join(adj, "user_idx", "left")
.select("user_idx", "Account", "Indirizzo_IP",
f.greatest(f.lit(0), f.col("visite_int") + f.coalesce(f.col("bump"), f.lit(0))).alias("visite_int"))
)
)
```

```

users = users.withColumn("user_type",
f.when(f.col("visite_int")==1, "new").when(f.col("visite_int").between(2,5), "returning").when(f.col("visite_int")>5, "power").otherwise("any")
)

```

### 3. STEP 2 – Generazione delle sessioni

Per ogni utente con almeno una visita si generano tante righe quante sono le visite. Ogni riga diventa una sessione distinta.

Appendice tecnica – codice:

```

sessions = (
users.filter("visite_int > 0")
.withColumn("session_num", f.explode(f.sequence(f.lit(1), f.col("visite_int"))))
.select("user_idx", "Account", "Indirizzo_IP", "user_type", "session_num")
)

```

### 4. STEP 3 – Assegnazione del journey per sessione

I pattern di journey vengono espansi per tipo utente, normalizzati per peso e poi assegnati a ogni sessione con una selezione a roulette. Si calcola anche il numero di azioni da generare per la sessione.

Appendice tecnica – codice:

```

patterns_df = spark.createDataFrame(
[(n,p,w,mi,ma,ut,d) for (n,p,w,mi,ma,ut,d) in JOURNEY_PATTERNS],
["name", "pages", "weight", "min_act", "max_act", "user_type", "desc"]
)
user_types = spark.createDataFrame([( "any", ), ( "new", ), ( "returning", ), ( "power", )], ["session_user_type"])
patterns_expanded = (patterns_df
.join(user_types, (patterns_df.user_type == "any") | (patterns_df.user_type == user_types.session_user_type))
.select(patterns_df["*"], user_types.session_user_type)
)

w = Window.partitionBy("session_user_type")
patterns_expanded = (patterns_expanded
.withColumn("total_w", f.sum("weight").over(w))
.withColumn("norm_w", f.col("weight")/f.col("total_w"))
)

sessions = sessions.withColumn("u", f.rand(555))
join_df = (sessions
.join(patterns_expanded, sessions.user_type == patterns_expanded.session_user_type, "inner")
.withColumn("cum_w", f.sum("norm_w").over(Window.partitionBy("user_idx", "session_num").orderBy("name")))
.where(f.col("u") <= f.col("cum_w"))
.dropDuplicates(["user_idx", "session_num"])
.select(
sessions["*"],
f.col("name").alias("journey_name"),
f.col("pages").alias("page_template"),
"min_act", "max_act"
)
)

tpl_size = f.size("page_template")
join_df = (join_df
.withColumn("num_actions",
f.when(
f.col("journey_name")=="full_purchase",
tpl_size
).otherwise(
f.when(f.col("min_act")==f.col("max_act"), f.col("min_act"))
.otherwise(f.col("min_act") + f.floor(f.rand(888) * (f.col("max_act") - f.col("min_act") + 1)))
).cast("int")
)
.drop("min_act", "max_act")
)

```

## 5. STEP 4 – Esplosione azioni e mappatura pagine

Si trasformano le sessioni in azioni/pagine, una riga per ogni azione utente.

Appendice tecnica – codice:

```
actions = (join_df
.withColumn("action_num", f.explode(f.sequence(f.lit(1), f.col("num_actions"))))
.withColumn("template_size", f.size("page_template"))
.withColumn("Pagina_Web",
f.when(
f.col("action_num") <= f.col("template_size"),
f.element_at("page_template", f.col("action_num"))
).otherwise(
f.when(f.col("journey_name")=="full_purchase",
f.element_at("page_template", f.col("template_size"))
).otherwise(
f.element_at("page_template", f.col("template_size"))
)
)
)
)
.select("user_idx", "Account", "Indirizzo_IP", "session_num", "journey_name", "Pagina_Web", "action_num")
)
```

## 6. STEP 5 – Anonimizzazione (IP coerente per sessione)

Alcune sessioni vengono rese anonime: l'IP viene rigenerato in modo deterministico e l'Account viene oscurato.

Appendice tecnica – codice:

```
anon_hash = f.abs(f.crc32(f.concat_ws("|", f.lit("anon"), f.col("user_idx").cast("string"),
f.col("session_num").cast("string"))))
anon_ip = f.concat_ws(".",
f.lit(10),
((anon_hash % 254) + 1).cast("int"),
((f.floor(anon_hash / 254) % 254) + 1).cast("int"),
((f.floor(anon_hash / 64516) % 254) + 1).cast("int")
)
actions = (actions
.withColumn("is_anon", f.rand(43) < f.lit(P_ANON))
.withColumn("Indirizzo_IP", f.when(f.col("is_anon"), anon_ip).otherwise(f.col("Indirizzo_IP")))
.withColumn("Account", f.when(f.col("is_anon"), f.lit(None)).otherwise(f.col("Account")))
.drop("is_anon")
)
```

## 7. STEP 6 – Timestamp e durate

Si assegna un istante di partenza casuale alla sessione e una durata specifica per ogni pagina.

Appendice tecnica – codice:

```
seconds_back = f.lit(DAYS_BACK*86400)
actions = (actions
.withColumn("session_start_sec", (f.rand(48)*seconds_back).cast("long"))
.withColumn("session_start_unix", f.unix_timestamp() - f.col("session_start_sec"))
)

duration_map = {
"Home": (8, 90),
"Search": (10, 120),
"Category": (15, 150),
"Product": (25, 300),
"Cart": (20, 180),
"Checkout": (45, 360),
"Help": (30, 240),
"Blog": (40, 300)
}
dur_expr = None
for pg, (mn, mx) in duration_map.items():
cur = f.round(f.lit(mn) + f.rand(100 + hash(pg) % 50) * (mx - mn), 2)
if dur_expr is None:
dur_expr = f.when(f.col("Pagina_Web") == pg, cur)
else:
dur_expr = dur_expr.when(f.col("Pagina_Web") == pg, cur)
Durata_Visita_Sito = dur_expr.otherwise(f.lit(30.0))

actions = (actions
.withColumn("Durata_Visita_Sito", Durata_Visita_Sito)
.withColumn("Data_Invio_Statistiche",
f.to_date(f.from_unixtime(f.col("session_start_unix") + (f.col("action_num")-1)*f.col("Durata_Visita_Sito"))))
.drop("session_start_sec", "session_start_unix")
)
```

## 8. STEP 7 – Oggetto\_Visionato e coerenza carrello/checkout

Si assegna un oggetto principale di sessione e si forza Cart/Checkout a usare uno degli oggetti realmente cercati nella stessa sessione. Si calcola anche il tempo passato sul singolo oggetto.

Appendice tecnica – codice:

```
obj_hash = f.abs(f.crc32(f.concat_ws("|", f.col("user_idx").cast("string"), f.col("session_num").cast("string"))))
session_obj_idx = (obj_hash % len(objects)).cast("int")
session_obj = f.element_at(objects_arr, session_obj_idx + f.lit(1))

actions = (actions
.withColumn(
"Oggetto_Visionato",
f.when(
f.col("Pagina_Web").isin("Product", "Category", "Cart", "Checkout", "Search"),
f.when(
f.col("Pagina_Web").isin("Product", "Category", "Search"),
f.when(f.rand(202)<0.80, session_obj)
.otherwise(f.element_at(objects_arr, (f.rand(203)*len(objects)).cast("int")+1))
)
)
.otherwise(session_obj)
)
)

w_sess = Window.partitionBy("user_idx", "session_num")
actions = (actions
.withColumn(
"searched_set",
f.array_distinct(
f.collect_set(
f.when(
f.col("Pagina_Web").isin("Product", "Search", "Category"),
f.col("Oggetto_Visionato")
)
)
).over(w_sess)
)
)

searched_size = f.size("searched_set")
rand_idx = (f.floor(f.rand(204)*f.greatest(searched_size, f.lit(1))) + 1).cast("int")
chosen_from_searched = f.element_at("searched_set", rand_idx)

actions = (actions
.withColumn(
"Oggetto_Visionato",
f.when(
f.col("Pagina_Web").isin("Cart", "Checkout"),
f.when(searched_size > 0, chosen_from_searched).otherwise(session_obj)
).otherwise(f.col("Oggetto_Visionato"))
)
.withColumn(
"Tempo_Visita_Su_Oggetto",
f.when(
f.col("Oggetto_Visionato").isNotNull(),
f.round(f.least(f.col("Durata_Visita_Sito")*(0.25+f.rand(47)*0.65), f.col("Durata_Visita_Sito")), 2)
).otherwise(f.lit(0.00))
)
.drop("searched_set")
)
)
```

## 9. STEP 8 – ID\_Sessione deterministico

Si costruisce una chiave basata su Account/anon, IP e numero sessione e si applica crc32.

Appendice tecnica – codice:

```
session_key = f.concat_ws("|",
f.coalesce(f.col("Account"), f.concat(f.lit("anon#"), f.col("user_idx"))),
f.col("Indirizzo_IP"),
f.col("session_num").cast("string")
)
actions = actions.withColumn("ID_Sessione", f.abs(f.crc32(session_key)).cast("bigint"))
```

## 10. STEP 9 – Output finale su Delta

Si preparano le colonne finali e si scrive la tabella Delta partizionata per Data\_Invio\_Statistiche.

Appendice tecnica – codice:

```
out_df = (actions
.select(
"ID_Sessione",
"action_num",
"journey_name",
f.col("Pagina_Web").cast("string").alias("Pagina_Web"),

```

```
f.col("Account").cast("string").alias("Account"),
f.col("Durata_Visita_Sito").cast("decimal(10,2)").alias("Durata_Visita_Sito"),
f.col("Oggetto_Visionato").cast("string").alias("Oggetto_Visionato"),
f.col("Tempo_Visita_Su_Oggetto").cast("decimal(10,2)").alias("Tempo_Visita_Su_Oggetto"),
f.col("Data_Invio_Statistiche").cast("date").alias("Data_Invio_Statistiche"),
f.col("Indirizzo_IP").cast("string").alias("Indirizzo_IP")
)
.repartition(8, "Data_Invio_Statistiche")
)

(out_df.write
.format("delta")
.mode("overwrite")
.option("overwriteSchema", "true")
.partitionBy("Data_Invio_Statistiche")
.saveAsTable("default.row_data_site"))
```